Întrebări = teorie = : Subiectul II

1. Dați 4 modalități de îmbunătățire a backtrackingului
   - alegerea variabilelor în altă ordine
   - îmbunătățirea restricțiilor
   - evitarea redundanțelor în spațiul de căutare

RESTRICȚII

2. Asemănări și deosebiri Greedy vs DP

Asemănare: rezolvă problema principală, împărțind-o în subprobleme (Divide et impera)

Greedy	DP
-> sunt menținute sol. parțiale din care evoluează stările parțiale urm. -> soluțiile parțiale ant. sunt eliminate -> necesită un criteriu optim -> conduce la optim local -> algoritm top-down	-> se păstrează sol. parțiale -> la construcția unei sol. noi, orice soluție parțială construită ult. anterior poate contribui -> nu necesită criteriu optim -> conduce la optim global -> algoritm bottom-up

DP: bottom up

3. Cum se poate modifica Floyd-Warshall ca să calc închiderea tranzitivă

R: - Se folosind o matrice cu valori de tip boolean (true / false). Dacă există muchie de la i la j valoarea va fi 1, altfel 0.

În loc să folosim operatori aritmetici, vom folosi operatori logici, astfel '+' este înlocuit cu '&&' și min cu '||'

-> mat[i][j] = mat[i][j] || [mat[i][k] && mat[k][j]

4. Numărul maxim și minim de CTC-uri într-un graf cu n noduri:

Nr. max: n (graful conține doar noduri izolate)

Nr. min: 1 (ciclu)

5. La algoritm se obține dacă $h(u) = 0$ la A^*?

R: Dijkstra

6. 2 euristici pt problema aranjării a 8 pătrate:

h_1 = nr. pătratelor a căror poziție diferă de cea finală

h_2 = suma distanțelor Manhattan între poziția curr. și poziția finală pt. fiecare dintre cele 8 pătrate.

7. Cum arată graful pt. problema cu fete și băieți ce tip de graf e și cum se rezolvă cuplajul

→ Un graf bipartit, o mulțime de noduri pt fete, o mulțime de noduri pt băieți, fetele în stânga, băieții în dreapta. Există muchie între fata x și băiatul y dacă x vrea să danseze cu y

- Pentru a afla nr. maxim de perechi pe care le putem forma ~~graf bipartit~~ facem o rețea de flux pe baza acestui legând toate nodurile din stânga de o sursă, toate nodurile din dreapta spre o destinație, iar absolut toate muchiile au capacitatea 1

8. DP vs. Divide et impera

a. D.e.i.
- top-down (subprobl)
- o subproblemă poate fi calculată de mai multe ori

b. DP:
- bottom-up (se combină sol. mai mici pt a obține soluții mari)
- o problemă nu va fi calculată de mai multe ori

9. Cum ați folosi schema minimax pentru un joc de șah

Se construiește arborele stărilor, până la o adâncime anume, apoi aleg mutările examinate mai întâi cu ajutorul euristicilor

Ex: • mutările cu care se iau piese
• mai întâi mutări cu scor mai bun
• mutările care au mai generat tăieri

10. Enumerați utilizările folosite în algoritmi de la curs ale tipilor de debut și de finish de la DFS.

-> • sortare topologică
• Kosaraju (CTC)
• Tarjan (biconexe)

11. Las Vegas vs. Monte Carlo

-> timp predet. de rulare
-> soluția calculată este garantat corectă dar nu optimală

-> timp determinat
-> cu cât îl lași să ruleze mai mult, cu cât obții o soluție mai apropiată de cea corectă

12. Explicați utilitatea alg. lui Johnson, de ce este mai bun decât alternativele

Sol: • Roy Floyd $O(n^3)$
• $n \times$ Dijkstra ($O(m \cdot n \log n)$)
• Bellman-Ford de la un nod fictiv

13. Cum modifici Dijkstra dacă graful e aciclic și orientat

Când graful este orientat, aciclic, putem sorta topologic graful și să calculăm același vector de distanțe ca la Dijkstra parcurgând nodurile din sortarea topologică de la stg. la dreapta și updatând vecinii distanțele:
d[vecin] = min(d[vecin], d[nod] + cost(nod, vecin))
d[sursă] = 0, d[noduri] = ∞;

14. Ce înseamnă o euristică admisibilă la A*?
R: ca $0 \leq h(u) \leq h^*(u)$, $\forall u$
Ce s-ar întâmpla dacă nu e admisibilă?
R: A* ar găsi o soluție dar nu se garantează optimalitate

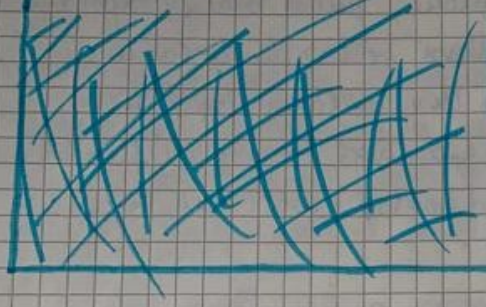
15. Caracteristicile Divide et Impera $T(x^n)$

Caracteristicile la 8,

$x^n = x^{n/2} \cdot x^{n/2}$, dacă n par
$x^n = x \cdot x^{n-1}$, n impar

16. BFS vs. A*

BFS	vs.	A*
• mixit next state based on heuristic function $f(u) = h$ with lowest heuristic value. Conteaza doar o stare cu cea mai mica heurist.		• mixit next state based on heuristic $f(u) = h + g$, h = aceeasi insemnatate ca la BFS dar g este costul de la starea initiala la o anumita stare, astfel nu alege cea mai mica euristica dar cea care alege cea mai mica euristica de a ajunge la acea stare



17. Dati ex. de un graf cu n noduri $m > n$ arce si o parcurgere in adancime care genereaza n arbori de parcurgere a grafului



Aplic DFS de la nodul 8 la nodul 1

18. Cum se modifica nr de CTC la adaugarea de muchii?
Nr. de CTC poate sa ramana la fel sau sa scada la adaugarea unei muchii, nu poate sa creasca. Nr. acesta scade daca avem de ex. un graf sub forma de lant si muchia inchide ciclul

19. Transformând un algoritm făcut la curs, scrieți în pseudocod un alg de detectare eficientă a existenței ciclurilor într-un graf orientat.
Ce complexitate are?

• O modificare Bellman-Ford: cost -1 pe fiecare muchie din graf, se pornește Bellman-Ford, dacă găsim un ciclu de cost negativ => există ciclu în graf
O(N * m)

20. Cum se poate transforma o problemă de determinare a fluxului maxim într-o rețea cu mai multe surse și mai multe destinații într-o problema de determinare a fluxului maxim într-o rețea cu o sursă și destinație?

R: Se construiește o sursă nouă pe care o legăm cu capacitate INF de toate sursele inițiale, și o destinație nouă, pe care o legăm cu capacitatea INF de toate dest. inițiale.

21. În ce cazuri complexitatea căutării drumurilor de cost minim într-un graf orientat G=(N,A) cu costuri pozitive este O(N+A)

R: Când graful este aciclic, deoarece avem cel mult un drum între oricare 2 noduri și atunci costul este unic determ.

22. A* vs BFS

BFS e particularizare Best First, A* e un alg. de căutare euristică

23. Nr maxim și minim de CTC netriviale într-un graf cu n noduri și a arce.

24. Pe ce caz Fold-Fulkerson merge prost ?

25. Ce este o cale reziduală și cum se obține? Care este capacitatea reziduală a căii? Exemplificați:

Presupunem că avem o rețea de flux. Graful rezidual este un graf care conține doar muchiile nesaturate. O cale reziduală este un drum care pleacă din sursă de la flux și merge până în destinație doar pe muchii nesaturate

Dacă presupunem drumul $v_1, \ldots, v_k$ unde v_1 = sursă v_k = destinație, atunci capacitatea reziduală a căii este $\min(cap[v[i+1]][v[i]] - flux[v[i+1]][v[i]])$, $i = 1, k-1$

26. Ce tipuri de muchii se pot întâlni într-un graf neorientat?

Fiind un graf neorientat, se pot găsi într-o parcurgere DFS 2 tipuri de muchii, muchii de avans (de arbore) folosite pentru explorarea nodurilor și muchii de întoarcere care unesc un nod cu un strămoș al său în arborele DF.

Graf orientat:
-> (muchii) arce de arbore: care conduc la descoperirea de vârfuri noi
-> arce de revenire: care leagă un vârf cu un strămoș în arborele DFS
-> arce de înaintare: care leagă un vârf cu un descendent în arborele DFS
-> arce de traversare: care leagă vârfuri din arbori DFS diferiți

27. Care este nr. total de relaxări de muchii făcute de Bellman-Ford? Explicați motivul.

Nr. total de relaxări de de muchii făcute de Bellman-Ford este $(n-2)*m$. Motivul pt care algoritmul folosește acest nr. de relaxări de muchii este că dacă vrem să detectăm existența unui ciclu negativ, atunci acest ciclu poate conține cel mult N noduri. Înainte să facem Bellman-Ford considerăm doar potențialele cicluri de lungime 2, apoi după K iterații luăm în calcul potențialele cicluri de lungime maxim K+2, deoarece K trb să fie N-2. La final se mai face o iterație prin care se încearcă relaxarea muchiilor și în caz că se găsește una care încă mai updatează o distanță, înseamnă că există ciclu negativ

28. Cum se poate modifica alg lui Dijkstra dacă graful e aciclic și orientat? Vezi (13)

29. Dacă facem *2 la costuri pe muchii sau +2 se modifică drumul minim găsit de Dijkstra?

Dacă facem +2, drumul rămâne același, dar i se dublează costul

Dacă adăugăm +2, se modifică drumul

1->3 : 4 — inițial drumul era 1->2->3

1->2 : 2 — După adăugarea +2 => drumul

2->3 : 1 — devine 1->3

30. Ce se întâmplă cu alg. minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din $\beta < \alpha$ în $\beta - k < \alpha$, unde k, constantă oarecare

1. $\beta = k$ => alg nu se schimbă
2. $\beta - k < \beta$ => riscăm să facem tăieturi inutile => rezultate greșite în urma rulării alg.
3. $\beta - k > \beta$ => riscăm să omitem tăieri ce ne-ar duce la creșterea timpului de rulare al alg, însă funct. acestuia va rămâne

31. Să se spună ce algoritm făcut la curs poate determina eficient care este drumul de lungime maximă într-un graf aciclic orientat cu lungimi pe arce și să se schițeze ideea folosirii acestuia și modificările necesare pt. rezolvare?

Sortare topologică pt. a găsi sursa + DFS modificat din sursa care întoarce distanța maximă calculată pe parcurs

32. A* vs DFS

Sunt folosiți pt. a obține un drum de la o sursă la destinație. A* caută un drum de cost minim, DFS caută un drum între cele 2 noduri, DFS inițializează aleatoriu iar A* relaxează muchiile în mod optim prin intermediul unei euristici

33. De ce Dijkstra merge doar pe costuri pozitive

Când un nod este marcat ca "closed", algoritmul a găsit (cea mai scurtă) cel mai scurt, și nu va mai căuta alt drum, se presupune că drumul găsit este cel mai scurt.

Pentru costuri negative

1 -> 3 : 2
1 -> 2 : 5
2 -> 3 : -10

Dijkstra găsește A -> C și dă fail când găsește A -> B -> C

34. Fie un graf neorientat cu n noduri și a arce. Care este nr. maxim / minim de punți și puncte de articulație al acestui graf? Justificați.

Nr. maxim de puncte de articulație: $n-2$

• -> • -> • -> • graf -> lanț

Nr maxim de punți: $n-1$

Nr. minim puncte de artic: 0 (nu = mucluri)

Nr. minim de punți: 0

35. Îmbunătățirea alg. de backtracking
• alegerea (Vezi ①)

37. Modificați un alg. de la curs, a.î. să detecteze dacă un graf e conex sau nu.

Facem BFS sau DFS, dacă toate nodurile sunt negre => graf conex, altfel nu.

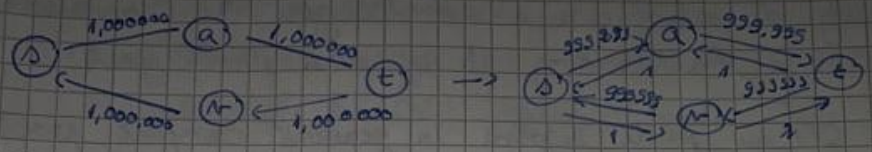
38. AC_3 vs. AC_1

$AC_1 = O(m \cdot n \cdot a^3)$

$AC_3 = O((n \cdot a)^2)$

AC_3 face mai puține ~~ale~~ apeluri ale funcției Revise decât AC_1. Dacă o constrângere aduce modificări în rețea AC_3, adaugă pt reverificare nodurile care pun diaza către modul de plecare al restricției verificate, iar la AC_1 se aplică alg. de consistență nodurilor și apoi se aplică REVISE până nu se mai realizează nicio schimbare.

39. Ford Fulkerson care merge prost



Se repetă de 999.999 ori

40. Care sunt pașii principali care trebuie urmați pt a rezolva o problemă folosind DP? Exemplificați (parantezarea)

- Descompunerea în subprobleme
  -> încercăm să definim subprobleme identice cu sub-problema originală, dar de dim. mai mică.

-> $\forall 1 \le i \le j \le n$

- $A_{ij} = A_i \times A \dots \times A_j$, A_{ij} are p_i linii și p_j col
- $m[i,j]$ = nr optim de înmulțiri pt. a rezolva subproblema $A_{i,j}$
- $s[i,j]$ = poziția primei paranteze pt subprobl. A_{ij}

-> problema inițială $A_{1,n}$

- Combinarea subproblemelor
  - pt $A_{i,j}$ trebuie găsit indicele $i \le k \le j$ care asigură parantezarea optimă
  $A_{i,j} = (A_{i..k} \times A_k) \times (A_{k+1}, \dots, A_j)$
  $A_{i,j} = A_{i,k} \times A_{k+1,j}$

- Alegerea optimală: căutăm optimul dintre toate var. posibile, pt asta trebuie ca și probl fol să aibă sol optime

41. Scrieți diferențele principale între backtracking și alg. de satisfacere a constrângerilor? Cum putem combina CSP și backtracking pt rez unei probl grele

CSP nu poate rezolva în general complet problema dată. Metoda ajută la limitarea spațiului de căutare. (ex. imp. în cond în care backtrack are compl. exponențială în cazul în care propagarea restricțiilor nu rezolvă problema se folosește backtracking pt generarea sol parțiale și propagarea restricțiilor după fiecare pas de backtracking pt a limita spațiul de căutare

În cazul Sudoku, CSP sunt fol la evitarea redundanțelor în spațiul de căutare.

42. Ce este o muchie înapoi? Dem că un graf aciclic nu poate să conțină muchii înapoi:

O muchie de la un nod în curs de evaluare la un alt nod în curs de evaluare.

Nu are voie să conțină muchii înapoi ptc. înseamnă că în cursul DFS se găsesc muchii către noduri care au fost deja evaluate -> ∃ ciclu

43. Când complexitatea căutării drumurilor de cost minim într-un graf orientat G(N, A) cu costuri ≥0 este O(N+A)

- Pt graf aciclic orientat (DAG) se face sort topo și apoi dist se pot calcula cu DFS -> O(V+E)
- Când toate costurile dintr-un graf sunt egale, se aplicăm BFS pt cost min.

44. MINIMAX + îmbunătățiri

```
minimax (node, depth, maxPlayer)
    if (depth = 0 || node is term. node)
        return the heuristic value of node;
    if ( maxPlay. ~~bestvalue~~ )
        value := -∞
        for each child of node do
            value := max (value, minimax (child, d-1, fals)
        return value
    else
        value := +∞
        for each child of n do
            val := min (value, minimax (child, d-1, True)
        return fo value
```

- Sortarea mutărilor ce vor fi analizate
- Limitarea adâncirii căutării

45. Calea reziduală ???

46. A* vs BFS

BFS se mișcă pas cu pas, mai departe care e urm. nod din graf și verifică dacă a ajuns unde trb. A* se folosește de euristică care dacă este aleasă corect duce mult mai rapid la rezolv. problemei

(Deseori BFS caută în zig-zag, A* se duce direct)

47. Algoritmi folosiți dacă ar trb. să calc cel mai mare nr. prim
Las Vegas (a ordare de tip)

48. Noțiunea de dominanță pt. alg A^*, explicând de ce este importantă în alegerea euristicii
Un alg A_1^* domină un alg A_2^* dacă orice nod expandat de A_1^* este expandat și de A_2^*

Ex: jocul 8-pătrățele
Euristici: h_1 = nr. de pătrățelelor a căror poziție curentă diferă de cea finală
h_2 = distanța Manhattan = suma dist pe vert și oriz dintre poz curentă și poziția finală (o h_2 mai informată decât h_1)

49. Exemplu de alg care poate det. cicluri de cost neg. într-un graf orientat.
Floyd Warshall, dacă pe diag principală apar val negative => cicluri negative

50. DFS pt clasificare, cum se poate det tipul fiecărei muchii din graf folosind o sg. parcurgere în adâncime:
Muchie de înaintare (nod curs -> nod ner),
muchie de întoarcere (nod curs -> nod curs)
muchie de traversare (nod curs -> nod evaluat)
cu Tarjan

51. Care e înălțimea maximă la care poate ajunge un nod în cadrul alg. de tip preflux fol. pt determinarea fluxului maxim.

Înălțimea max: $2 \cdot |V| - 1$, lung maximă pos. a unui drum rezidual

52. Dându-se matricea finală de distanțe obținută folosind Floyd-Warshall se poate folosi dacă graful are ciclu negativ?

În mod normal matricea are 0 pe diagonală, în cazul unui ciclu negativ, pe diag. pot apărea valori diferite de 0, deoarece dacă plec dintr-un nod și urmez ciclul, la final voi ajunge în același nod cu un cost < 0, fiind ciclu negativ, se înlocuiește 0 cu costul negativ

53. Dem că un graf are cicluri ($\Leftrightarrow$) în timpul unei parcurgeri DFS sunt desc arce noi

DFS $\Rightarrow$ cycle: în timpul parcurgerii DFS se găsește muchie către un nod în curs de vizitat (gri), există o muchie la un nod în curs de vizitare $\Leftrightarrow$ există ciclu

cycle $\Rightarrow$ DFS: există ciclu $\Rightarrow$ există măcar o muchie care se întoarce la un nod "gri", dacă la un nod în curs de vizitare $\Rightarrow$ parcurgerea DFS va găsi acea muchie

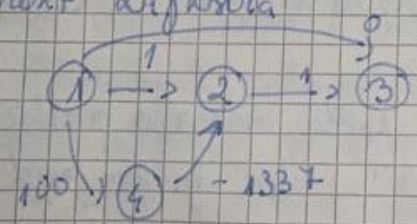
54. Ce cu e imp. ca euristica folosită de A^* să fie admisibilă. Dem că orice euristică consistentă este admisibilă.

A^* este un algoritm pentru găsirea de drumuri și traversarea grafurilor, căutând o cale directă ~~ca~~ între 2 noduri. A^* are o performanță mai bună ~~când~~ folosind o euristică care să îl ghideze

Pt. ca o euristica să fie admisibilă, ea trebuie să nu supraestimeze costul atingerii scopului. Dacă se întâmplă asta, A^* poate ignora drumul cel mai scurt pt. că valoarea supraestimată a drumului inițial pare mai mare decât altă posibilă

O euristică consistentă estimează mereu distanța de la un nod vecin la destinație ca fiind mai mică sau egală cu costul real. În acest caz -> euristica ei e admisibilă.

55. Graf cu 5 noduri, muchie cost neg, calculează greșit Dijkstra



56. Tăieturi alfa - beta reduc complexitatea unei rez. minimax pt jocuri doar dacă sunt fol cu o altă tehnică

-> Nu mai generăm anumite ramuri când știm că minimax nu o vor lua pe acolo -> reducem timpul de calcul

-> Folosirea unei euristici care să parcurgă subarbori care garantează o tăiere $\alpha\beta$

Tăieturi $\alpha - \beta$

Acest alg încearcă să optimizeze mini / nega max:
pe parcursul examinării arborelui de soluții se pot elimina întregi subarbori corespunzători unei mișcări m, dacă pe parcursul analizei găsim că mișcarea m este mai slabă calitativ decât mișcarea curentă.

Johnson:

-> returnează o matrice ce conține costurile drumurilor minime pentru toate perechile de vârfuri resp. FALSE dacă ∃ ciclu de cost negativ

Algoritmul adaugă un nou vârf, unit prin toate celelalte cu arce de cost 0 și folosește mai apoi Bellman-Ford pt. a înlătura toate costurile negative din graf. Se aplică apoi Dijkstra pornind din fiecare vârf din graful inițial

$O(VE \log V)$